

moodtune

Project Documentation — Architecture, Implementation, and How It Works

August 2026

Contents

1. Overview	2
2. How It Works (User Flow)	2
3. Architecture	3
4. Tech Stack	3
5. Project Structure	4
6. Backend Deep Dive	5
6.1 Entry point (src/index.js)	5
6.2 Routes	5
6.3 Services	6
6.4 Caching (lib/cache.js)	8
6.5 Error handling	8
7. Frontend Deep Dive	9
7.1 App.jsx — state and orchestration	9
7.2 Components	9
7.3 useAudioPreview hook	10
7.4 utils/color.js	11
8. Key Design Decisions	11
9. Environment Variables	12
10. Running Locally	12
11. Deployment	13
12. Known Limitations / Not Yet Implemented	13
13. API Reference	14

1. Overview

moodtune turns an album cover into a small mood board. A person searches for an artist, album, or song — or drops in their own image — and the app pulls a color palette out of the artwork, turns that palette into a short mood label (“Cool & Moody”, “Warm & Nostalgic”), and surfaces similar artists and songs to keep exploring, each with a hoverable audio preview.

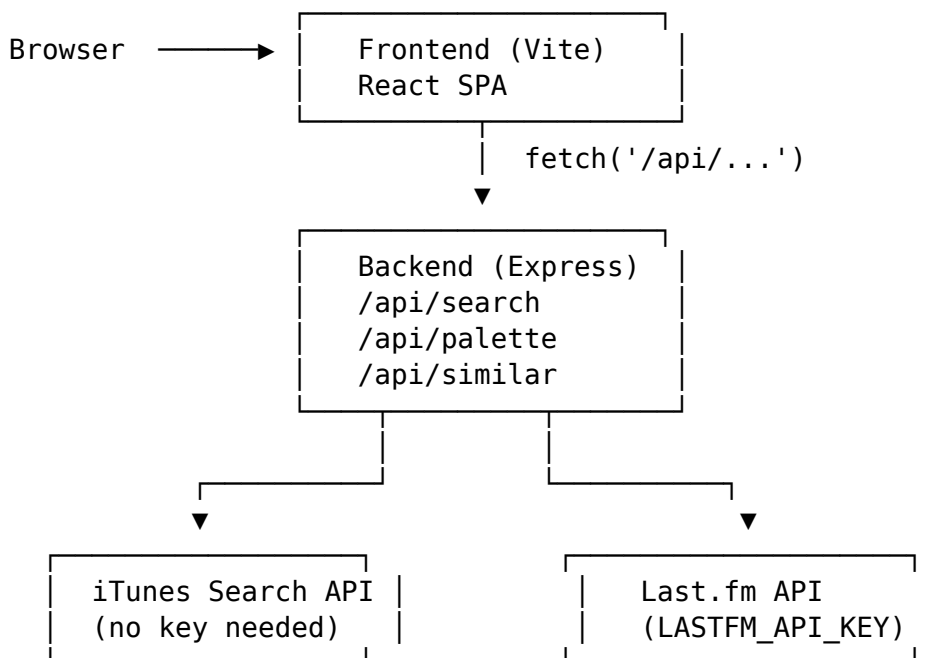
It’s a two-part application: a small Node/Express API that talks to iTunes and Last.fm and does the image analysis, and a React/Vite single-page frontend that renders the board and drives all of the interaction.

Backend	Node.js, Express 5
Frontend	React 19, Vite 8
Styling / animation	Tailwind CSS v4, Framer Motion
Color extraction	node-vibrant (server-side)
Data sources	iTunes Search API (no key), Last.fm API (free key)
State/persistence	None — stateless per request, in-memory cache only

2. How It Works (User Flow)

1. **Find a starting point.** Type into the search box (artists, albums, and songs from iTunes, interleaved) or drag an image onto the upload zone.
 2. **The board populates in two parallel steps:**
 - The cover art is sent to the backend, which extracts a palette and derives a mood label.
 - The artist name is sent to Last.fm for a similar-artists list, each enriched with artwork and (where available) a previewable track.
 3. **Explore the palette.** Eight swatches are shown, sized by nothing but click-to-copy convenience — clicking one copies its hex code, with a checkmark confirmation.
 4. **Keep exploring.** Two tabs sit under the mood board: **Artists** and **Songs**. Clicking an artist or a song loads it as the new board, so browsing chains indefinitely — a similar artist becomes the new center, with its own palette and its own similar artists.
 5. **Preview a song.** Hovering an album cover with a preview available (the main board’s artwork, or a thumbnail in the Songs tab) fades in a 30-second clip; moving the mouse away fades it back out rather than cutting it abruptly.
 6. **Uploads without metadata.** If the image came from an upload rather than a search result, the app doesn’t know who the artist is — it asks, so it can still fetch similar artists.
-

3. Architecture



The two services are deployed independently (see §11): the frontend as a static build, the backend as a small always-on Node process. They communicate purely over HTTP — the frontend never talks to iTunes or Last.fm directly.

Why the backend does the color extraction, not the browser. Pulling a cross-origin image into a browser `<canvas>` for pixel reads taints the canvas under CORS rules, which blocks exactly the operation needed (reading pixel data back out). Rather than working around that per-image, the backend fetches the raw bytes itself and runs the color quantizer there — this works identically whether the art came from a search result’s URL or a user’s uploaded file.

Why there’s no database. Every request is self-contained: given a search query, palette request, or artist name, the response is fully derivable from what iTunes/Last.fm return plus a short-lived in-memory cache (see §6.4). There’s nothing that needs to persist between requests or survive a restart.

4. Tech Stack

Backend (backend/)

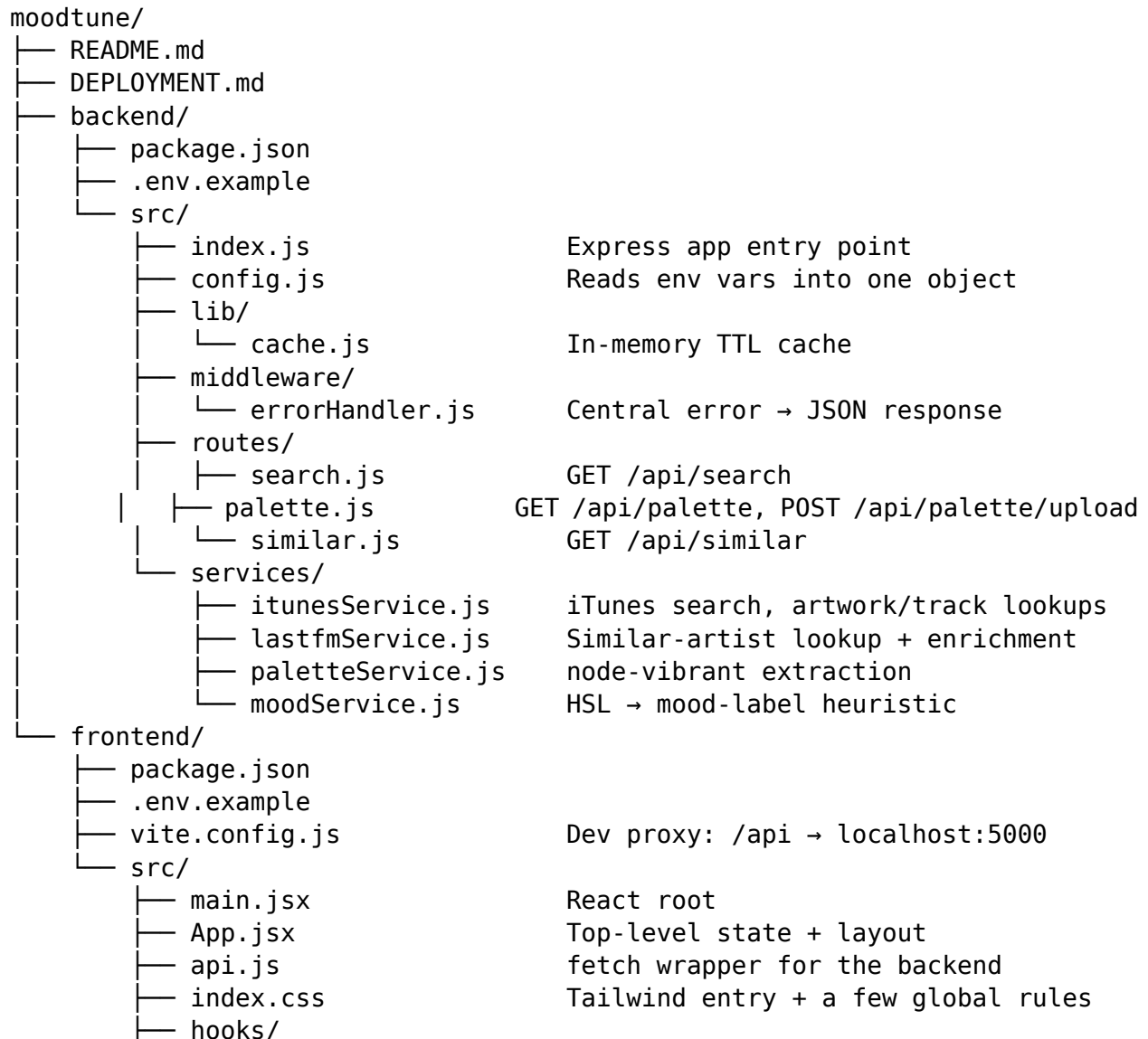
- **express** — HTTP server and routing
- **cors** — restricts which origins may call the API (`CORS_ORIGIN`)
- **multer** — parses the multipart upload for the “upload your own image” flow, in-memory (no disk writes)
- **node-vibrant** — color quantization (MMCQ algorithm) against raw image bytes

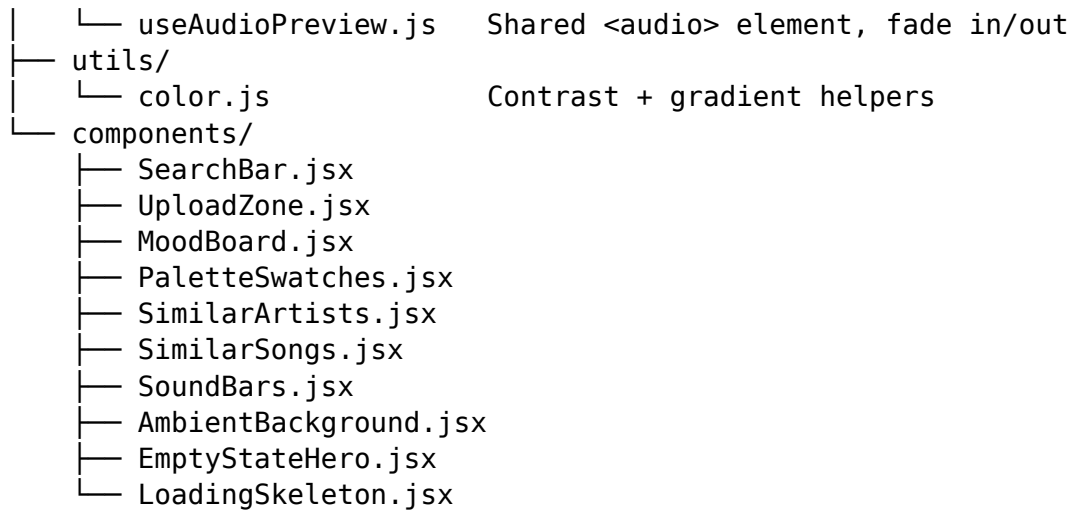
- **dotenv** — loads `.env` into `process.env` in development

Frontend (frontend/)

- **react / react-dom** — UI
 - **vite** — dev server and production bundler
 - **tailwindcss** (v4, via `@tailwindcss/postcss`) — utility-first styling
 - **framer-motion** — hover/tap gestures, enter/exit transitions, layout animation (the shared tab-indicator pill under Keep Exploring)
 - No state management library, no router — a single view, state held in `App.jsx` via `useState`
-

5. Project Structure





6. Backend Deep Dive

6.1 Entry point (src/index.js)

A standard Express app: CORS is configured from CORS_ORIGIN (comma-separated list, so multiple frontend origins can be allowed at once), JSON body parsing is enabled, and three routers are mounted under /api. A catch-all 404 handler and a central error handler (§6.5) sit at the end of the middleware chain.

```

app.use(cors({ origin: config.corsOrigin.split(",").map((o) => o.trim()) }));
app.use(express.json());

app.get("/api/health", (req, res) => {
  res.json({ ok: true, lastfmConfigured: Boolean(config.lastfmApiKey) });
});

app.use("/api/search", searchRouter);
app.use("/api/palette", paletteRouter);
app.use("/api/similar", similarRouter);

```

/api/health exists so both local setup and a fresh deploy can be sanity-checked with a single request — it reports whether a Last.fm key is configured without leaking the key itself.

6.2 Routes

Method	Path	Query / Body	Returns
GET	/api/health	—	{ ok, lastfmConfigured }

Method	Path	Query / Body	Returns
GET	/api/search	?q=<text>	{ results: [...] } — albums and tracks interleaved
GET	/api/palette	?imageUrl=<url>	{ swatches, moodLabel }
POST	/api/palette/upload	multipart field image	{ swatches, moodLabel }
GET	/api/similar	?artist=<name>	{ results: [...] } — enriched similar artists

Each route is a thin adapter: validate the query/body, call into a service, forward the error to errorHandler on failure. All the actual logic lives in services/.

6.3 Services

itunesService.js — search, artwork, and track lookups

Wraps the iTunes Search API (<https://itunes.apple.com/search>), which needs no API key but does enforce an undocumented per-IP rate limit (roughly 20 requests/minute, returned as a bare 403).

- **searchAlbums / searchTracks** — free-text search, used directly by the search box. Tracks and albums share the same shape (artworkUrl, artist, etc.) plus a kind discriminator, so the frontend can render them interchangeably; a track additionally carries a previewUrl when iTunes has a 30-second clip for it.
- **searchMusic** — runs both searches in parallel and interleaves the two result lists, so songs don't get buried under a page of albums.
- **findArtworkForArtist / findTopTrackForArtist** — used to enrich Last.fm's similar-artist results, which don't reliably include images. Both search with attribute: artistTerm (biasing the match toward the artist field specifically) and then verify the result's artist name against the one being searched for before trusting it:

```
export async function findArtworkForArtist(artistName) {
  return cached(key, 30 * 60 * 1000, async () => {
    const results = await searchAlbums(artistName, 5, { attribute: "artistTerm" });
    const target = normalizeName(artistName);
    const match = results.find((r) => normalizeName(r.artist) === target);
    return match?.artworkUrl ?? null;
  });
}
```

Free-text search will happily return an album by a *different* artist if the query text matches its title better than it matches the artist field — this verification step trades a small amount of “coverage” (some legitimate artists get null back) for never showing the wrong artist's face under someone else's name.

lastfmService.js — similar artists

Calls Last.fm's `artist.getsimilar` and enriches every result with a thumbnail and, separately, a previewable track (for the Songs tab). Two things worth calling out:

- **Last.fm's placeholder images are filtered out.** Last.fm stopped hosting real per-artist photos for most entries and instead returns a URL to one specific generic "star" image. It's a real, loadable URL, so a naive "does an image URL exist" check would accept it — the code filters it out by its known filename hash so the iTunes fallback actually runs:

```
const LASTFM_PLACEHOLDER_IMAGE_ID = "2a96cbd8b46e442fc41c2b86b821562f";
```

- **Enrichment is concurrency-limited, not parallel-unbounded.** Each similar artist now needs up to two iTunes lookups (artwork + a track), and firing all of them at once for a page of ~10 artists risks tripping iTunes' rate limit in a single burst. A small worker-pool helper processes a handful at a time:

```
async function mapWithConcurrency(items, limit, fn) {
  const results = new Array(items.length);
  let next = 0;
  async function worker() {
    while (next < items.length) {
      const i = next++;
      results[i] = await fn(items[i], i);
    }
  }
  await Promise.all(Array.from({ length: Math.min(limit, items.length) }, worker));
  return results;
}
```

Since each individual lookup is itself cached for 30 minutes, this cost is only paid once per artist regardless of how many times it's requested afterward.

paletteService.js — color extraction

Runs `node-vibrant` against raw image bytes (either fetched from a URL or received as an upload buffer) and returns the top 8 swatches by population — that is, by how much of the image each color actually covers, not a curated set of named roles:

```
const vibrant = Vibrant.from(buffer).build();
await vibrant.getPalette();
const colors = vibrant.result?.colors ?? [];

const swatches = [...colors]
  .sort((a, b) => b.population - a.population)
  .slice(0, 8)
  .map((swatch) => ({ hex: swatch.hex, population: swatch.population }));
```

`node-vibrant`'s higher-level `.getPalette()` API only exposes 6 curated named swatches (`Vibrant`, `Muted`, `LightVibrant`, etc.), which are chosen for contrast/legibility rather than frequency — they can surface a color that barely appears in the image

while skipping one that dominates it. Reading `vibrant.result.colors` directly (available once a full Vibrant instance is built, not just the palette builder) exposes the complete, unfiltered output of the underlying MMCQ quantizer.

moodService.js — palette → mood label

A deterministic heuristic, not a model call: the same palette always produces the same label, and it needs no external API or network round-trip.

1. Convert each swatch's hex to HSL.
2. Average hue, saturation, and lightness across all swatches — weighted by each swatch's population (how much of the image it covers), so a sliver of background color doesn't pull the average as hard as a color filling half the cover.
3. Map the averaged hue to a first word (Fiery, Warm, Sunny, Fresh, Cool, Moody, Dreamy, Mysterious, Romantic) via hue bands.
4. Map averaged saturation/lightness (each bucketed low/mid/high) to a second word via a 3×3 lookup table (Intense, Vibrant, Playful, Brooding, Balanced, Airy, ...).
5. Join them: "Cool & Moody".

6.4 Caching (lib/cache.js)

A minimal in-memory Map with per-entry TTLs — no Redis, no external dependency:

```
export async function cached(key, ttlMs, fn) {  
  const hit = cacheGet(key);  
  if (hit !== undefined) return hit;  
  const value = await fn();  
  cacheSet(key, value, ttlMs);  
  return value;  
}
```

Every outbound call to iTunes or Last.fm is wrapped in `cached(...)`, with TTLs tuned per endpoint (searches: 10 min, similar-artist lists: 15 min, artist artwork/track lookups: 30 min, palette extraction: 30 min). This is what keeps the app comfortably under both APIs' rate limits during normal use — it isn't shared across server instances and resets on every restart, which is an accepted tradeoff for an app with no database.

6.5 Error handling

Services throw plain Error objects, occasionally with a `.status` attached (e.g. 429 for a rate-limit, 422 for "couldn't extract a palette from that image"). A single Express error-handling middleware turns any of those into a JSON response with the right status code, and logs anything that reaches a 5xx to the server console:

```
export function errorHandler(err, req, res, next) {  
  const status = err.status || 500;  
  if (status >= 500) console.error(err);  
}
```



```
res.status(status).json({ error: err.message || "Something went wrong." });
}
```

7. Frontend Deep Dive

7.1 App.jsx — state and orchestration

All board state lives in one component, in one object:

```
function emptyBoard() {
  return {
    artworkUrl: null, title: "", subtitle: "",
    artistName: null,           // null = unknown (upload flow before identification)
    swatches: [], moodLabel: "",
    similarArtists: [],
    previewUrl: null,          // only set for a song search result
  };
}
```

Three entry points can produce a new board — a search selection, an uploaded file, or clicking a similar artist/song — and each does the same two things in parallel: kick off a palette request (`loadPaletteFromUrl` / the upload equivalent) and a similar-artists request (`loadSimilar`), each with its own loading flag so the palette and the “keep exploring” section can resolve independently rather than blocking on each other.

One deliberate non-fallback: when a similar-artist pick has no verified artwork (see `findArtworkForArtist` in §6.3), the frontend shows an explicit placeholder rather than falling back to the *previous* board’s artwork — showing no image is preferable to showing a plausible-looking but wrong cover under the new artist’s name.

7.2 Components

Component	Responsibility
<code>SearchBar.jsx</code>	Debounced (350ms) search-as-you-type against <code>/api/search</code> ; renders a dropdown of album/song results
<code>UploadZone.jsx</code>	Drag-and-drop / click-to-browse image upload
<code>MoodBoard.jsx</code>	The main card: artwork, title/subtitle, mood label, palette, and the Keep Exploring tabs. Owns the hover-preview interaction on the main artwork
<code>PaletteSwatches.jsx</code>	Renders the 8 swatches; click-to-copy hex, with computed text contrast so the hex label stays legible on any background

Component	Responsibility
<code>SimilarArtists.jsx</code>	Horizontal-scroll list of similar artists (Artists tab)
<code>SimilarSongs.jsx</code>	Horizontal-scroll list of one representative track per similar artist (Songs tab), each independently hover-previewable
<code>SoundBars.jsx</code>	Small reusable “now playing” equalizer animation, used by both <code>MoodBoard</code> and <code>SimilarSongs</code>
<code>AmbientBackground.jsx</code>	Full-viewport soft blobs recolored from the current palette, so the mood extends past the card itself
<code>EmptyStateHero.jsx</code>	Idle-state illustration shown before any board has loaded
<code>LoadingSkeleton.jsx</code>	Pulse-placeholder skeletons for the palette and similar-artists sections while their requests are in flight

Keep Exploring tabs. `MoodBoard.jsx` holds `exploreTab` state (“artists” | “songs”) and renders a shared, animated pill behind whichever tab is active using `Framer Motion`’s `layoutId`, so the indicator slides between tabs instead of snapping.

Progressive thumbnail fade-in. Both `SimilarArtists` and `SimilarSongs` fade each thumbnail in individually once its image has actually decoded (`onLoad`), rather than swapping from a blank/skeleton state to the image the instant the `<img>` tag mounts — network images arrive at different speeds, and a hard cut on a slow one reads as the UI stalling.

The pointer-events-none overlay pattern. Every hover-preview overlay (main artwork in `MoodBoard`, thumbnails in `SimilarSongs`) is a `pointer-events-none` div whose opacity is driven by React state, not CSS `:hover`. It has to be state-driven because a non-interactive element can’t receive its own hover events — and it has to be `pointer-events-none` specifically so this purely visual layer never intercepts real interactions with the image underneath it (right-click “save image as”, drag-to-desktop), even while sitting at `opacity-0` directly on top of it.

7.3 useAudioPreview hook

One shared `<audio>` element per hook instance, reused across hovers rather than creating a new element each time (which would risk overlapping clips):

```
audio.play().catch(() => setPlayingUrl(null));
fadeVolume(audio, 1, reduceMotion ? 0 : FADE_MS);
```

`fadeVolume` ramps `audio.volume` from its current value to a target over `FADE_MS` (350ms) using `requestAnimationFrame`, for both fade-in (on hover) and fade-out (on un-hover) — so playback never starts or stops abruptly. Two details matter here:

- **Clamping.** `audio.volume` throws a `RangeError` for any value outside `[0, 1]`, and floating-point drift during the ramp can nudge a value a hair past either edge. Every write is wrapped in `Math.min(1, Math.max(0, ...))` to guarantee it never does.
- **Interruption handling.** If a fade is interrupted mid-ramp (a quick re-hover), the next fade continues smoothly from wherever the volume currently sits rather than jumping — `cancelAnimationFrame` on the in-flight ramp plus reading `audio.volume` as the new starting point handles this.

`stop()` is exposed so callers can kill playback explicitly on events the hook itself can't observe — e.g. `MoodBoard` calls it whenever the displayed artwork changes, since swapping the underlying image out from under an already-stationary cursor doesn't fire a `mouseleave`.

7.4 `utils/color.js`

Two small, pure helpers with no dependencies:

- **`readableTextColor(hex)`** — computes WCAG relative luminance for a background hex and returns whichever of near-black/near-white reads better on top of it. Used for swatch hex labels, which sit directly on top of arbitrary generated colors.
- **`gradientFromSwatches(hexes)`** — builds a CSS linear-gradient string from up to 4 swatches, for the mood board's background.

8. Key Design Decisions

A short list of the non-obvious choices, gathered in one place:

- **Palette extraction happens server-side**, purely to avoid the browser canvas CORS-taint restriction on cross-origin images (§3) — it isn't a security or performance choice, just the only place this can run at all for search-result artwork.
- **Palette is 8 raw dominant swatches, not 6 curated "roles"** — a more literal reflection of what's actually in the artwork (§6.3, `paletteService.js`).
- **Mood labeling is a deterministic HSL heuristic**, not a model call — same palette always yields the same label, no external dependency, no latency.
- **Similar-artist enrichment is concurrency-limited (3 at a time)**, not unbounded `Promise.all`, specifically to stay under iTunes' undocumented rate limit when enriching a full page of results (§6.3).
- **Artist-name verification before trusting iTunes art**, even at the cost of sometimes returning no image, rather than risk showing a wrong artist's cover (§6.3, §7.1).
- **Audio previews fade rather than cut**, with explicit volume clamping to avoid a silent-failure `RangeError` (§7.3).
- **Hover-preview overlays are pointer-events-none and state-driven**, so an invisible layer never blocks real interaction with the image beneath it (§7.2).

- **No database, no auth, no persistence** — every response is derivable from the current request plus a short-lived cache; this keeps the whole app stateless and trivial to deploy as two independent services.

9. Environment Variables

File	Variable	Required	Notes
backend/.env	LASTFM_API_KEY	Yes	Free key from last.fm/api/account/create . App boots without it, but /api/similar returns a 503 until it's set
backend/.env	PORT	No	Defaults to 5000. Render/most hosts inject their own PORT automatically
backend/.env	CORS_ORIGIN	No	Defaults to http://localhost:5173 . Comma-separated for multiple allowed origins
frontend/.env	VITE_API_BASE_URL	Only in production	Base URL of the deployed backend. Unset in dev, since Vite's dev server proxies <code>/api</code> to <code>localhost:5000</code> (see <code>vite.config.js</code>). Baked in at build time — changing it requires a rebuild

10. Running Locally

```
# Backend
cd backend
cp .env.example .env      # add your LASTFM_API_KEY
npm install
```

```
npm run dev                # http://localhost:5000

# Frontend (separate terminal)
cd frontend
npm install
npm run dev                # http://localhost:5173
```

Vite's dev server proxies any `/api/*` request to `http://localhost:5000`, so the frontend code always just calls `fetch('/api/...')` with no environment-specific branching needed in development.

11. Deployment

The app deploys as two independent services — see `DEPLOYMENT.md` in the repo root for the full click-by-click walkthrough. In short:

1. **Backend → Render** (or any Node host): root directory backend, build `npm install`, start `npm start`, with `LASTFM_API_KEY` set as an environment variable.
 2. **Frontend → Vercel** (or any static host): root directory frontend, Vite auto-detected, with `VITE_API_BASE_URL` set to the backend's deployed URL.
 3. **CORS**: once both are live, the backend's `CORS_ORIGIN` is set to the frontend's actual deployed URL.
-

12. Known Limitations / Not Yet Implemented

- **No automated test suite.** Verification so far has been manual, plus targeted checks of specific interaction bugs (audio fade clamping, hover pointer-events, animation timing) during development.
 - **No accessibility/contrast audit pass** across the full range of moods a palette can generate — `readableTextColor` handles the swatch labels specifically, but a systematic pass hasn't been done.
 - **No export, theming, or history.** Ideas floated but not built: export the current board as an image, a light/dark toggle, and a session history of past boards.
 - **In-memory cache only** — fine for a single small instance, but it resets on every restart/redeploy and wouldn't be shared across multiple instances if the backend were ever scaled horizontally.
 - **Render's free tier sleeps after 15 minutes idle**, adding roughly a minute of cold-start latency to the first request after inactivity — a hosting-tier characteristic, not an app bug.
-

13. API Reference

GET /api/health

No parameters. Returns { ok: true, lastfmConfigured: boolean }.

GET /api/search?q=<text>

Returns { results: [...] }. Each result is either:

```
{ "id": 123, "kind": "album", "artist": "...", "album": "...", "artworkUrl": "..."  
  ↪ }
```

or

```
{ "id": 456, "kind": "song", "artist": "...", "album": "...", "track": "...",  
  ↪ "artworkUrl": "...", "previewUrl": "..." }
```

GET /api/palette?imageUrl=<url>

Returns:

```
{  
  "swatches": [{ "hex": "#3a2f4d", "population": 8213 }, "... up to 8"],  
  "moodLabel": "Cool & Moody"  
}
```

POST /api/palette/upload

Multipart form, field name image (max 8MB, image MIME types only). Same response shape as /api/palette.

GET /api/similar?artist=<name>

Returns { results: [...] }, each entry:

```
{  
  "name": "...",  
  "match": 0.87,  
  "imageUrl": "... or null",  
  "topTrack": { "artist": "...", "track": "...", "artworkUrl": "...",  
    ↪ "previewUrl": "..." } // or null  
}
```